

jTime

Metodologie di Programmazione

Andrea Castelli, Febbraio 2026

Indice

1. Introduzione.....	2
2. Funzionalità Implementate.....	2
2.1 Gestione Progetti.....	2
2.2 Gestione Attività.....	3
2.3 Pianificazione.....	3
2.4 Report e Statistiche.....	4
3. Responsabilità Individuate.....	4
3.1 Model.....	4
3.2 View.....	5
3.3 Controller.....	5
3.4 Persistence.....	6
4. Architettura delle Classi.....	6
4.1 Entità Model.....	6
4.2 Controller.....	7
4.3 View.....	7
4.4 Livello di persistenza e interfacce.....	7
5. Organizzazione Dati e Persistenza.....	8
5.1 Schema Database.....	8
5.2 Tecnologie di Persistenza.....	9
6. Estendibilità del Sistema.....	9
6.1 Modifica alle entità.....	9
6.2 Sistema di Tag.....	9
6.3 Multi-utente.....	9
6.4 Sistema di backup.....	10
6.5 Schedulazione oraria.....	10
6.6 Inserimento multiplo.....	10
6.7 Versione mobile / web.....	10
6.8 Sistema di notifiche.....	10
6.9 Sistema di refresh automatico.....	10
7. Manuale d'uso.....	10
8. Conclusioni.....	11

1. Introduzione

Il sistema **jTime** rappresenta un'applicazione utile all'organizzazione e divisione delle varie attività giornaliere, sia sul posto di lavoro che per quanto riguarda la vita privata. L'obiettivo del progetto è quello di creare un sistema unico di agenda in grado di organizzare al meglio i propri impegni.

Il sistema consente di:

- Creare e organizzare progetti
- Aggiungere attività con stime temporali
- Pianificare le giornate evitando sovraccarichi
- Analizzare le performance attraverso report dettagliati

L'intero sistema è stato sviluppato in Java seguendo il pattern MVC, con l'aggiunta di css per la parte grafica.

Per la parte grafica ho utilizzato **JavaFX** mentre per la persistenza ho scelto **Hibernate/JPA**. Inizialmente avevo considerato anche altre opzioni (tipo file XML), ma alla fine ho optato per un approccio database perché più scalabile per eventuali sviluppi futuri.

2. Funzionalità Implementate

2.1 Gestione Progetti

Ho implementato due schermate principali per gestire i progetti. Il Project Controller si occupa di validare i dati e salvarli attraverso il Project Repository.

Creazione Progetti

La prima schermata permette di creare nuovi progetti inserendo:

- Nome del progetto
- Descrizione (campo opzionale per dare più contesto)
- Stato iniziale (parte sempre come "Attivo")

Nella schermata è presente una tabella che mostra tutti gli ultimi progetti inseriti e permette di eliminare quelli creati per errore.

Chiusura Progetti

L'altra schermata permette di visualizzare i progetti creati (filtrandoli anche per id, nome, attivi e completati), incluse le task assegnate ad essi. Per chiudere un progetto è necessario che tutte le attività collegate ad esso siano terminate.

2.2 Gestione Attività

Formata anch'essa da due pagine, utilizza l' `ActivityController` per validare i dati e l'`ActivityRepository` per salvarli.

Creazione Attività

Il form per la creazione di un'attività prevede:

- Nome attività
- Descrizione dettagliata
- Stima del tempo necessario (in formato HH:MM)

Le attività hanno come durata minima 5 minuti e massima 24 ore, con unità minima di misura 5 minuti.

La schermata inoltre permette di eliminare le attività create per errore , a patto che non siano associate , completate o pianificate. Per facilitare la visualizzazione è presente sia una colonna nella tabella che un filtro.

Queste operazioni preparano i dati necessari alle schermate di pianificazione e report, che si basano sulle informazioni di stima, pianificazione e completamento

Modifica Attività

Questa schermata permette di gestire tutte le attività create e inserirle nei meccanismi del sistema.

La tabella centrale mostra tutte le attività (con possibilità di filtrarle per id ,nome , progetto e stato) e il form di destra consente di inserire/modificare i dati dell'attività scelta (nel caso non sia conclusa). I campi sono 3 e possono essere combinati tra di loro:

- Data pianificazione
- Progetto a cui deve essere associata (solo progetti attivi)
- Durata effettiva (che implica completamento dell'attività)

La durata effettiva ha gli stessi vincoli della stima mentre la pianificazione è valida solo se la somma delle stime delle attività programmate non supera le 24 ore (modificabile a 8 in caso di uso lavorativo).

Per sincronizzare i dati relativi ai progetti, funzione che verrà implementata in futuro, per ora è necessario cliccare sul tasto cerca oppure selezionare un'attività.

2.3 Pianificazione

La schermata di pianificazione offre una vista d'insieme sulle attività programmate.

La tabella mostra le attività per il giorno selezionato evidenziando sia le quantità che il dettaglio di quelle completate o non.

2.4 Report e Statistiche

Questa è l'ultima schermata, dedicata all'analisi dei dati e presenta dei filtri temporali e per progetto.

La schermata mostra viste differenti sui dati:

- totali ore effettuate e rimasti
- Grafico relativi a tutti i progetti creati
- Grafico relativo allo stato delle attività nel lasso di tempo selezionato

3. Responsabilità Individuate

L'utilizzo dell'architettura MVC permette di differenziare bene le responsabilità ed evitare di violare i principi SOLID

3.1 Model

Il livello Model contiene le due entità principali del sistema e, inoltre, le interfacce di repository specifiche che definiscono le operazioni di accesso ai dati offerte al resto del sistema.

Project

Rappresenta un progetto e include:

- Identificatore univoco (generato automaticamente)
- Nome e descrizione
- Stato ("Attivo" o "Completato")
- Lista di attività associate

Con i relativi getters e setters per ottenere le informazioni

Activity

Modella le attività con questi attributi:

- ID univoco
- Nome e descrizione
- Stima tempo (formato stringa "HH:MM")
- Durata effettiva
- Data di pianificazione
- Riferimento al progetto
- Flag "eliminabile"

Inclusi i getters e setters per l'accesso ai dati.

3.2 View

Per la vista ho utilizzato **JavaFX** con file **FXML**.

Ho creato un file FXML per ogni schermata principale:

- jTime_home.fxml - schermata principale con TabPane
- jTime_addTask.fxml - form creazione attività
- jTime_manageTask.fxml - gestione attività avanzata
- jTime_addProjects.fxml - creazione progetti
- jTime_closeProjects.fxml - chiusura progetti
- jTime_planning.fxml - vista pianificazione
- jTime_statistics.fxml - report e statistiche

Ogni FXML è completamente separato dalla logica di business. Questo significa che potrei sostituire JavaFX con un'altra tecnologia (tipo una webapp) senza toccare i controller o il model.

3.3 Controller

Qui ho due livelli di controller:

Controller delle View (JavaFX)

Ogni schermata ha il suo controller che gestisce gli eventi dell'interfaccia:

- HomeViewController : gestisce la creazione delle tab
- AddTaskController : gestisce l'inserimento delle nuove attività
- ManageTaskController : gestisce la selezione e guida all'inserimento dei dati richiesti
- AddProjectsController : gestisce la creazione di nuovi progetti
- CloseProjectsController : gestisce la visualizzazione e la chiusura dei progetti
- PlanningController : gestisce la visualizzazione e il filtraggio delle attività per mostrare le attività giornaliere pianificate
- StatisticsController : permette di visualizzare statistiche dettagliate sull'andamento generale

Questi controller non contengono business logic, si limitano a raccogliere i dati inseriti dall'utente e chiamare i vari business controller che eseguiranno le operazioni , per poi aggiornare la schermata.

Controller Business Logic

I controller di business ricevono e utilizzano i repository tramite le loro interfacce, mantenendo il livello di controllo indipendente dai dettagli della persistenza.

- **ActivityController**: valida sia l'inserimento che la modifica delle attività (creazione , pianificazione , completamento ed eliminazione)

- **ProjectController**: valida la creazione e la chiusura dei progetti
- **ReportController**: aggrega , filtra e calcola i dati necessari da mostrare nella schermata.

3.4 Persistence

Il livello di persistenza non espone direttamente le classi concrete dei repository al resto del sistema, ma solo le relative interfacce (ActivityRepositoryInterface e ProjectRepositoryInterface). In questo modo i controller di business lavorano contro un contratto astratto, mentre i dettagli di implementazione (Hibernate, query HQL, gestione delle sessioni) restano incapsulati nelle classi ActivityRepository e ProjectRepository. Questa scelta facilita la sostituzione o l'estensione del meccanismo di persistenza senza impattare la logica di dominio.

ProjectRepository

Gestisce tutte le operazioni CRUD sui progetti:

- save() - inserimento/aggiornamento
- findById() - ricerca per ID
- findAll() - lista completa
- findActive() e findCompleted() - query filtrate
- delete() - eliminazione

ActivityRepository

Operazioni simili per le attività, più alcune query specializzate:

- findByProject() - attività di un progetto specifico
- findByDate() - attività pianificate per una data
- findByDateRange() - range temporale
- findEliminabili() - solo eliminabili
- findByFilters() - ricerca avanzata con criteri multipli

4. Architettura delle Classi

4.1 Entità Model

La classe Activity rappresenta la base delle attività e contiene tutte le informazioni necessarie , tra cui il progetto a cui è stata assegnata in quanto un'attività potrà essere assegnata ad un unico progetto.

La classe Project contiene le informazioni basilari e la lista di tutte le attività ad esse collegate.

4.2 Controller

I controller business sono classi normali (POJO) che ricevono i repository di cui hanno bisogno.

Per esempio, `ActivityController` ha come dipendenze le interfacce `ActivityRepositoryInterface` e `ProjectRepositoryInterface`, che vengono implementate da `ActivityRepository` e `ProjectRepository`.

4.3 View

I file FXML definiscono la struttura XML delle interfacce. Ogni elemento ha un `fx:id` che lo collega al controller corrispondente.

Il controller JavaFX ha delle annotazioni `@FXML` che permettono di accedere a questi elementi dal codice.

4.4 Livello di persistenza e interfacce

Il livello di persistenza è stato progettato introducendo interfacce dedicate che astraggono l'accesso ai dati da parte della logica applicativa. In particolare:

- `CrudRepository<T, ID>` definisce le operazioni CRUD di base (`save`, `update`, `delete`, `findById`, `findAll`) in modo generico.
- `ActivityRepositoryInterface` estende questo contratto con le operazioni specifiche per le attività (`findByProject`, `findByDate`, `findByDateRange`, `findEliminabili`, `findByFilters`).
- `ProjectRepositoryInterface` estende il contratto CRUD con le operazioni specifiche per i progetti (`findByStato`, `findEmpty`, `findEliminabili`).

Le classi concrete `ActivityRepository` e `ProjectRepository` implementano rispettivamente `ActivityRepositoryInterface` e `ProjectRepositoryInterface`, occupandosi dei dettagli di accesso al database tramite Hibernate/JPA (creazione delle query, gestione delle transazioni, ecc.).

I controller di business (`ActivityController` e `ProjectController`) dipendono dalle interfacce dei repository, e non dalle implementazioni concrete. Questo permette di sostituire in futuro la tecnologia di persistenza (ad esempio da Hibernate a un altro ORM o a un servizio REST) senza modificare la logica applicativa, ma solo fornendo nuove implementazioni delle interfacce.

5. Organizzazione Dati e Persistenza

5.1 Schema Database

Il database è composto da due tabelle principali.

Tabella projects

Colonna	Tipo	Vincoli
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
nome	VARCHAR(255)	NOT NULL
descrizione	TEXT	-
stato	VARCHAR(50)	NOT NULL

Tabella activities

Colonna	Tipo	Vincoli
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT
nome	VARCHAR(255)	NOT NULL
descrizione	TEXT	-
stima_tempo	VARCHAR(5)	NOT NULL
durata_effettiva	VARCHAR(5)	-
data_pianificazione	DATE	-
progetto_fk	INTEGER	FOREIGN KEY → projects(id)
eliminabile	BOOLEAN	DEFAULT TRUE

5.2 Tecnologie di Persistenza

La persistenza è gestita dalle classi Repository che regolano tutte le azioni CRUD sul database dei dati precedentemente validati dai Controller.

I repository utilizzano Hibernate/JPA per tradurre le operazioni astratte (save, findById, ecc.) in query SQL sulle tabelle projects e activities.

6. Estendibilità del Sistema

Nella progettazione del sistema jTime sono stati considerati i principi SOLID per favorire manutenibilità ed estendibilità. In particolare, il livello di persistenza è stato astratto tramite interfacce dedicate (CrudRepository<T, ID>, ActivityRepositoryInterface, ProjectRepositoryInterface), in modo che i controller di business dipendano da contratti astratti e non dalle implementazioni concrete dei repository.

Questa scelta, insieme alla chiara separazione tra controller di view (JavaFX), controller di business e repository, rende il sistema facilmente evolvibile: è possibile aggiungere nuove funzionalità (come tag, multi-utente o notifiche) introducendo nuove entità, nuovi controller e nuove implementazioni di repository, senza dover modificare la logica esistente.

6.1 Modifica alle entità

Nella classe Activity potrebbe essere inserita una data che testimonia il limite massimo entro il quale può essere pianificata oppure un campo che mostri la priorità.

Nella classe Project potrebbero essere inseriti il campo priorità e la gestione delle date di inizio e di fine

6.2 Sistema di Tag

L'introduzione dei tag permetterebbe una migliore gestione delle attività e una maggiore leggibilità. Alcuni dei tag potrebbero riguardare la priorità, il contesto in cui svolgere le azione (online, casa, call) e collegamenti a delle attività padre. L'introduzione dei tag richiederebbe una nuova entità Tag, una tabella di associazione tra attività e tag, e un eventuale TagRepository che implementi CrudRepository, mantenendo allineata l'architettura di persistenza.

6.3 Multi-utente

Implementare le funzioni di login e registrazione utente permetterebbe di accedere in contemporanea a calendari privati e comuni così da gestire al meglio l'organizzazione ed evitare accavallamenti. Questa estensione richiederebbe una nuova entità User, un repository dedicato e l'aggiunta di un riferimento all'utente all'interno delle attività e dei progetti, senza modificare la logica dei controller esistenti, che filtrerebbero semplicemente i dati per utente.

6.4 Sistema di backup

Una cosa fondamentale per il sistema sarà l'introduzione di un sistema di backup che permetta di aggiungere fault tolerance e avere sempre una copia dei dati disponibile in caso di malfunzionamenti.

6.5 Schedulazione oraria

Ad ora il sistema permette solo di inserire le attività nel giorno desiderato ma non di suddividerle per orario , il che risulta molto utile nella pianificazione giornaliera. Inserendo un nuovo campo e dei nuovi controlli sarà facilmente implementabile. Inoltre questo porterà alla divisione di una task in più sotto task da una durata ridotta.

6.6 Inserimento multiplo

Possibilità di inserire eventi uguali ripetuti nel tempo così da ridurre il numero di azioni dell'utente necessarie per la calendarizzazione.

6.7 Versione mobile / web

Per facilitare l'utilizzo ed aumentare l'efficacia , sarà fondamentale per il sistema passare in versione mobile o web così da essere accessibile in qualsiasi momento. L'architettura del progetto facilita questo passaggio in quanto basterebbe cambiare la parte di view.

6.8 Sistema di notifiche

Altra aggiunta utile sarà il sistema di notifiche che darà al sistema il cambiamento necessario per diventare una vera e propria agenda. Alla pianificazione delle attività verrà agganciata una richiesta di inserimento promemoria a scelta dell'utente

6.9 Sistema di refresh automatico

Attualmente in alcune pagine risulta macchinoso aggiornare i dati che non vengono immediatamente caricati nelle schermate successive , per questo nelle prossime versioni verrà implementato un servizio di refresh automatico che permetterà a tutte le schermate di accedere ai dati in tempo reale.

7. Manuale d'uso

La prima schermata permette di inserire le attività all'interno del database. Ogni attività deve avere un nome obbligatorio e una descrizione opzionale. Possono essere eliminate solo quelle che non hanno altri dati (che verranno inseriti successivamente)

Nella seconda si possono inserire i progetti , anch'essi devono avere nome obbligatorio e descrizione opzionale. Anche per questa schermata i progetti eliminabili sono evidenziati e sono quelli che non hanno attività a essi collegate.

La terza schermata permette la gestione delle attività. E' possibile filtrare le attività per id , nome e stato. Successivamente nella tabella appariranno tutte le attività che rispettano i filtri selezionati e per essere sarà possibile inserire la data, nel caso la si voglia pianificare , il progetto , nel caso la si voglia assegnare , o le ore effettive di durata , nel caso la si voglia completare. Per le attività non completate al click sulla tabella verranno caricati tutti i dati e sarà possibile cambiarli.

La quarta schermata è dedicata alla chiusura dei progetti. Saranno visualizzati tutti i progetti con la possibilità di filtrarli. Al click su uno di essi verranno mostrate tutte le attività così da poter vedere quali sono completate e quali non. Un progetto può essere chiuso solo se tutte le attività ad esso assegnate sono state completate.

Nella quinta possiamo vedere le attività schedulate per giorno insieme a una statistica che riporta il numero di attività totali , completate e mancanti.

Nell'ultima schermata possiamo utilizzare i filtri per modificare il grafico delle attività e mostrare solo per un determinato periodo o progetto. Inoltre in fondo sono presenti delle previsioni orarie per la conclusione dei dati filtrati

8. Conclusioni

Il sistema jTime rappresenta un modo semplificato ma comunque dettagliato per gestire il proprio tempo. Inoltre grazie alla sua architettura è altamente personalizzabile e facilmente espandibile , rendendolo la base per molte possibili applicazioni più avanzate.